Complete Monitoring Stack Guide

Prometheus & Grafana Installation on AWS EC2 Linux

Version: 1.0
Date: August 2025
Author: Monitoring Stack Setup Guide

Table of Contents

Introduction

This guide provides a complete step-by-step process to set up a monitoring stack using Prometheus and Grafana on an AWS EC2 Linux instance. By the end of this guide, you will have:

- A fully functional Prometheus server collecting system metrics
- Node Exporter providing detailed system statistics
- Grafana dashboard server with beautiful visualizations
- Custom dashboards displaying your server metrics
- A secure, production-ready monitoring solution

Architecture Overview:

```
EC2 Instance (Linux)
├── Prometheus (Port 9090) - Metrics Collection & Storage
├── Node Exporter (Port 9100) - System Metrics Export
└── Grafana (Port 3000) - Visualization & Dashboards
```

Prerequisites

AWS Requirements

- AWS Account with EC2 access
- EC2 instance running Linux (Amazon Linux 2, Ubuntu 18.04+, CentOS 7+)
- SSH key pair for instance access
- Security group with appropriate inbound rules

Recommended EC2 Instance Specifications

- **Instance Type:** t3.medium or larger (2 vCPU, 4GB RAM)
- **Storage:** 20GB GP3 SSD minimum
- **Operating System:** Amazon Linux 2 or Ubuntu 20.04 LTS

Required Knowledge

- Basic Linux command line skills
- SSH connectivity
- Basic understanding of web services

EC2 Instance Setup

Step 1: Launch EC2 Instance

1. **Login to AWS Console**
   - Navigate to EC2 Dashboard
   - Click "Launch Instance"
2. **Choose AMI**
   - Select "Amazon Linux 2 AMI" or "Ubuntu Server 20.04 LTS"
3. **Choose Instance Type**
   - Select t3.medium (recommended) or t3.large for higher workloads

4. **Configure Instance**
   - Use default settings or customize as needed
   - Ensure instance has a public IP
5. **Add Storage**
   - Increase volume size to 20GB
   - Select GP3 for better performance
6. **Configure Security Group**
   - Create new security group with following rules:

Type	Protocol	Port Range	Source	Description
SSH	TCP	22	Your IP	SSH Access
HTTP	TCP	80	0.0.0.0/0	HTTP (Optional)
Custom TCP	TCP	9090	Your IP	Prometheus
Custom TCP	TCP	3000	Your IP	Grafana
Custom TCP	TCP	9100	Your IP	Node Exporter

7. **Launch Instance**
   - Select your key pair
   - Launch the instance

Step 2: Connect to Instance

```bash
# Connect via SSH
ssh -i your-keypair.pem ec2-user@your-public-ip

# For Ubuntu instances, use:
ssh -i your-keypair.pem ubuntu@your-public-ip
```

Step 3: Update System

```bash
```

```
# For Amazon Linux 2 / CentOS
sudo yum update -y

# For Ubuntu
sudo apt update && sudo apt upgrade -y
```

Prometheus Installation

Step 1: Create Prometheus User and Directories

bash

```
# Create prometheus system user
sudo useradd --no-create-home --shell /bin/false prometheus

# Create required directories
sudo mkdir /etc/prometheus
sudo mkdir /var/lib/prometheus

# Set directory ownership
sudo chown prometheus:prometheus /etc/prometheus
sudo chown prometheus:prometheus /var/lib/prometheus
```

Step 2: Download and Install Prometheus

bash

```
# Navigate to temp directory
cd /tmp

# Download latest Prometheus (check https://prometheus.io/download/ for latest version)
wget https://github.com/prometheus/prometheus/releases/download/v2.45.0/prometheus-2.45.0.linux-amd64.tar.gz

# Extract archive
tar xvf prometheus-2.45.0.linux-amd64.tar.gz

# Navigate to extracted directory
cd prometheus-2.45.0.linux-amd64

# Copy binaries to system directories
sudo cp prometheus /usr/local/bin/
sudo cp promtool /usr/local/bin/

# Set binary ownership
sudo chown prometheus:prometheus /usr/local/bin/prometheus
sudo chown prometheus:prometheus /usr/local/bin/promtool

# Copy console files
sudo cp -r consoles /etc/prometheus
sudo cp -r console_libraries /etc/prometheus

# Set console file ownership
sudo chown -R prometheus:prometheus /etc/prometheus/consoles
sudo chown -R prometheus:prometheus /etc/prometheus/console_libraries
```

Step 3: Create Prometheus Configuration

bash

```
# Create main configuration file
sudo nano /etc/prometheus/prometheus.yml
```

Add the following configuration:

yaml

```
# Global settings
global:
  scrape_interval: 15s         # How frequently to scrape targets
  evaluation_interval: 15s     # How frequently to evaluate rules

# Rule files (for alerting rules)
rule_files:
  # - "alert_rules.yml"
  # - "recording_rules.yml"

# Scrape configurations
scrape_configs:
  # Prometheus itself
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
    scrape_interval: 5s
    metrics_path: /metrics

  # Node Exporter (system metrics)
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['localhost:9100']
    scrape_interval: 5s

  # Add more jobs here as needed
  # - job_name: 'application'
  #   static_configs:
  #     - targets: ['localhost:8080']
```

bash

```
# Set configuration file ownership
sudo chown prometheus:prometheus /etc/prometheus/prometheus.yml
```

Step 4: Create Prometheus Systemd Service

bash

```
# Create systemd service file
sudo nano /etc/systemd/system/prometheus.service
```

Add the following service configuration:

```ini
[Unit]
Description=Prometheus Time Series Collection and Processing Server
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
Restart=always
RestartSec=5s
ExecStart=/usr/local/bin/prometheus \
  --config.file /etc/prometheus/prometheus.yml \
  --storage.tsdb.path /var/lib/prometheus/ \
  --web.console.templates=/etc/prometheus/consoles \
  --web.console.libraries=/etc/prometheus/console_libraries \
  --web.listen-address=0.0.0.0:9090 \
  --web.enable-lifecycle \
  --log.level=info

[Install]
WantedBy=multi-user.target
```

Step 5: Start Prometheus Service

```bash
```

```
# Reload systemd daemon
sudo systemctl daemon-reload

# Enable Prometheus service
sudo systemctl enable prometheus

# Start Prometheus service
sudo systemctl start prometheus

# Check service status
sudo systemctl status prometheus

# View logs if needed
sudo journalctl -u prometheus -f
```

Step 6: Verify Prometheus Installation

bash

```
# Test local access
curl http://localhost:9090

# Check if Prometheus is collecting metrics
curl http://localhost:9090/metrics

# Access web interface (replace with your public IP)
# http://your-ec2-public-ip:9090
```

Node Exporter Installation

Node Exporter provides detailed system metrics that Prometheus can scrape.

Step 1: Download and Install Node Exporter

bash

```
# Navigate to temp directory
cd /tmp

# Download Node Exporter
wget https://github.com/prometheus/node_exporter/releases/download/v1.6.0/node_exporter-1.6.0.linux-amd64.tar.g

# Extract archive
tar xvf node_exporter-1.6.0.linux-amd64.tar.gz

# Copy binary
sudo cp node_exporter-1.6.0.linux-amd64/node_exporter /usr/local/bin/

# Create node_exporter user
sudo useradd --no-create-home --shell /bin/false node_exporter

# Set binary ownership
sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter
```

Step 2: Create Node Exporter Service

bash

```
# Create systemd service file
sudo nano /etc/systemd/system/node_exporter.service
```

Add the following configuration:

ini

```
[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
Restart=always
RestartSec=5s
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
```

Step 3: Start Node Exporter Service

bash

```
# Reload systemd daemon
sudo systemctl daemon-reload

# Enable Node Exporter service
sudo systemctl enable node_exporter

# Start Node Exporter service
sudo systemctl start node_exporter

# Check service status
sudo systemctl status node_exporter
```

Step 4: Verify Node Exporter

bash

```
# Test local access
curl http://localhost:9100/metrics

# You should see system metrics like:
# node_cpu_seconds_total
# node_memory_MemAvailable_bytes
# node_filesystem_size_bytes
```

Grafana Installation

Step 1: Install Grafana Repository

For Amazon Linux 2 / CentOS / RHEL:

bash

```
# Create Grafana repository file
sudo tee /etc/yum.repos.d/grafana.repo << EOF
[grafana]
name=grafana
baseurl=https://rpm.grafana.com
repo_gpgcheck=1
enabled=1
gpgcheck=1
gpgkey=https://rpm.grafana.com/gpg.key
sslverify=1
sslcacert=/etc/pki/tls/certs/ca-bundle.crt
EOF

# Install Grafana
sudo yum install grafana -y
```

For Ubuntu / Debian:

bash

```
# Install required packages
sudo apt-get install -y software-properties-common wget

# Add Grafana GPG key
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -

# Add Grafana repository
echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee -a /etc/apt/sources.list.d/grafana.list

# Update and install
sudo apt-get update
sudo apt-get install grafana -y
```

Step 2: Start Grafana Service

```bash
# Reload systemd daemon
sudo systemctl daemon-reload

# Enable Grafana service
sudo systemctl enable grafana-server

# Start Grafana service
sudo systemctl start grafana-server

# Check service status
sudo systemctl status grafana-server
```

Step 3: Access Grafana Web Interface

1. **Open web browser and navigate to:**

   ```
   http://your-ec2-public-ip:3000
   ```

2. **Default login credentials:**
   - Username: `admin`
   - Password: `admin`
3. **You will be prompted to change the password on first login**

Connecting Grafana to Prometheus

Step 1: Add Prometheus Data Source

1. **Login to Grafana web interface**
2. **Navigate to Data Sources:**
   - Click on Configuration (gear icon) in left sidebar
   - Click "Data Sources"
3. **Add Data Source:**
   - Click "Add data source"
   - Select "Prometheus"
4. **Configure Prometheus Data Source:**
   - **Name:** `Prometheus`
   - **URL:** `http://localhost:9090`
   - **Access:** `Server (default)`
   - **HTTP Method:** `GET`
   - Leave other settings as default
5. **Test Connection:**
   - Click "Save & Test"
   - You should see "Data source is working" message

Step 2: Alternative Configuration via File

```bash
```

```
# Create datasource provisioning directory
sudo mkdir -p /etc/grafana/provisioning/datasources

# Create datasource configuration file
sudo tee /etc/grafana/provisioning/datasources/prometheus.yml << EOF
apiVersion: 1

datasources:
 - name: Prometheus
   type: prometheus
   access: proxy
   url: http://localhost:9090
   isDefault: true
   editable: true
   jsonData:
    httpMethod: GET
    prometheusType: Prometheus
    prometheusVersion: 2.45.0
EOF

# Restart Grafana to load configuration
sudo systemctl restart grafana-server
```

Creating Custom Dashboards

Step 1: Import Pre-built Dashboards

Import Node Exporter Dashboard:

1. **Navigate to Import:**
   - Click "+" in left sidebar
   - Click "Import"
2. **Import by ID:**
   - Enter Dashboard ID: `1860`
   - Click "Load"
3. **Configure Import:**
   - Name: `Node Exporter Full`
   - Select "Prometheus" as data source
   - Click "Import"

Additional Recommended Dashboards:

Dashboard ID	Name	Description
1860	Node Exporter Full	Complete system metrics
3662	Prometheus 2.0 Overview	Prometheus server metrics
12486	Node Exporter Quickstart	Simplified system dashboard
11074	Node Exporter for Prometheus	Alternative comprehensive dashboard

Step 2: Create Custom Dashboard

1. **Create New Dashboard:**
   - Click "+" in left sidebar
   - Click "Dashboard"
   - Click "Add new panel"
2. **Basic Panel Configuration:**
   - **Panel Title:** "CPU Usage"
   - **Query:** `100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`
   - **Legend:** "CPU Usage %"
3. **Panel Settings:**
   - **Visualization:** Time series
   - **Unit:** Percent (0-100)
   - **Min:** 0
   - **Max:** 100
4. **Save Panel:**
   - Click "Apply"

Step 3: Essential Metrics Dashboard

Create a comprehensive dashboard with the following panels:

Panel 1: CPU Usage

```promql
```

```
# Query
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

# Settings
- Title: CPU Usage
- Unit: Percent (0-100)
- Visualization: Stat or Time series
```

Panel 2: Memory Usage

```promql
# Query
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100

# Settings
- Title: Memory Usage
- Unit: Percent (0-100)
- Visualization: Gauge
```

Panel 3: Disk Usage

```promql
# Query
100 - ((node_filesystem_avail_bytes{mountpoint="/",fstype!="rootfs"} / node_filesystem_size_bytes{mountpoint="/",fsty

# Settings
- Title: Disk Usage (Root)
- Unit: Percent (0-100)
- Visualization: Gauge
```

Panel 4: Network I/O

```promql
```

```
# Query for Receive
rate(node_network_receive_bytes_total{device!="lo"}[5m])

# Query for Transmit
rate(node_network_transmit_bytes_total{device!="lo"}[5m])

# Settings
- Title: Network I/O
- Unit: Bytes/sec
- Visualization: Time series
```

Panel 5: Load Average

```
promql

# Query
node_load1
node_load5
node_load15

# Settings
- Title: Load Average
- Legend: 1m, 5m, 15m
- Visualization: Time series
```

Panel 6: Uptime

```
promql

# Query
node_time_seconds - node_boot_time_seconds

# Settings
- Title: System Uptime
- Unit: Seconds
- Visualization: Stat
```

Step 4: Save Dashboard

1. **Save Dashboard:**
   - Click save icon (disk icon) at top
   - Enter name: "System Overview"

- Add description: "Comprehensive system monitoring dashboard"
- Click "Save"

2. **Set as Home Dashboard (Optional):**
   - Go to Dashboard settings (gear icon)
   - Check "Set as home dashboard"

Dashboard Examples

Example 1: System Performance Dashboard

Dashboard Structure:

Example 2: Application Monitoring Dashboard

If you're monitoring applications, add these panels:

HTTP Request Rate:

```promql
rate(http_requests_total[5m])
```

Response Time:

```promql
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
```

Error Rate:

```promql
rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) * 100
```

Example 3: Advanced System Dashboard

Disk I/O:

```promql
# Read IOPS
rate(node_disk_reads_completed_total[5m])

# Write IOPS
rate(node_disk_writes_completed_total[5m])

# Read Throughput
rate(node_disk_read_bytes_total[5m])

# Write Throughput
rate(node_disk_written_bytes_total[5m])
```

Process Count:

```promql
node_procs_running
node_procs_blocked
```

File System Details:

```promql
# Available space by mount point
node_filesystem_avail_bytes{fstype!="tmpfs"}

# Inode usage
node_filesystem_files_free{fstype!="tmpfs"}
```

Troubleshooting

Common Issues and Solutions

Issue 1: Prometheus Not Starting

Symptoms:

- Service fails to start
- Connection refused on port 9090

Solutions:

```bash
# Check service status
sudo systemctl status prometheus

# Check logs
sudo journalctl -u prometheus -n 50

# Validate configuration
/usr/local/bin/promtool check config /etc/prometheus/prometheus.yml

# Check file permissions
ls -la /etc/prometheus/
ls -la /var/lib/prometheus/

# Fix permissions if needed
sudo chown -R prometheus:prometheus /etc/prometheus/
sudo chown -R prometheus:prometheus /var/lib/prometheus/
```

Issue 2: Node Exporter Not Providing Metrics

Symptoms:

- Prometheus shows Node Exporter target as down
- No system metrics available

Solutions:

```bash
```

```
# Check Node Exporter status
sudo systemctl status node_exporter

# Test Node Exporter locally
curl http://localhost:9100/metrics

# Check firewall (if enabled)
sudo firewall-cmd --list-all

# Verify Prometheus configuration
grep -A 5 "node-exporter" /etc/prometheus/prometheus.yml
```

Issue 3: Grafana Cannot Connect to Prometheus

Symptoms:

- Data source test fails
- "Data source is not working" error

Solutions:

bash

```
# Test Prometheus from Grafana server
curl http://localhost:9090/api/v1/targets

# Check Prometheus targets page
# Navigate to http://your-ip:9090/targets

# Verify Grafana logs
sudo journalctl -u grafana-server -n 50

# Check network connectivity
netstat -tlnp | grep :9090
```

Issue 4: Dashboard Shows No Data

Symptoms:

- Dashboard panels show "No data"
- Queries return empty results

Solutions:

1. **Check Time Range:**
   - Ensure dashboard time range includes data
   - Try "Last 5 minutes" or "Last 1 hour"
2. **Verify Metrics:**

```bash
# Check if metrics exist in Prometheus
curl 'http://localhost:9090/api/v1/query?query=up'
```

3. **Check Query Syntax:**
   - Verify PromQL query syntax
   - Test queries in Prometheus web interface
4. **Verify Data Source:**
   - Ensure correct data source is selected
   - Test data source connection

Issue 5: High Resource Usage

Symptoms:

- High CPU or memory usage
- Slow response times

Solutions:

```bash
```

```
# Monitor resource usage
top
htop
iostat -x 1 5

# Check Prometheus storage size
du -sh /var/lib/prometheus/

# Optimize Prometheus configuration
# Reduce scrape frequency in prometheus.yml
scrape_interval: 30s  # Instead of 15s

# Configure retention policy
--storage.tsdb.retention.time=7d  # Keep data for 7 days
```

Performance Optimization

Prometheus Optimization

Storage Optimization:

bash

```
# Add to prometheus.service ExecStart line
--storage.tsdb.retention.time=15d \
--storage.tsdb.retention.size=10GB \
--storage.tsdb.wal-compression
```

Query Optimization:

- Use recording rules for complex queries
- Limit query time ranges
- Use appropriate aggregation functions

Grafana Optimization

Configuration Optimizations:

ini

```
# In /etc/grafana/grafana.ini
[dashboards]
min_refresh_interval = 5s

[users]
viewers_can_edit = false

[auth.anonymous]
enabled = false
```

Security Best Practices

Network Security

1. **Security Group Configuration:**
   - Restrict source IPs to your organization
   - Use specific IP ranges instead of 0.0.0.0/0
   - Consider VPN access for remote monitoring
2. **Firewall Configuration:**

```bash
# Enable firewall (if not already enabled)
sudo systemctl enable firewalld
sudo systemctl start firewalld

# Allow specific ports
sudo firewall-cmd --permanent --add-port=9090/tcp
sudo firewall-cmd --permanent --add-port=3000/tcp
sudo firewall-cmd --permanent --add-port=9100/tcp
sudo firewall-cmd --reload
```

Application Security

1. **Change Default Passwords:**
   - Always change Grafana admin password
   - Use strong passwords (12+ characters)
2. **Enable HTTPS:**

```bash
```

```
# Generate self-signed certificate (for testing)
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/grafana/grafana-selfsigned.key \
  -out /etc/grafana/grafana-selfsigned.crt

# Update grafana.ini
sudo nano /etc/grafana/grafana.ini
```

ini

```
[server]
protocol = https
cert_file = /etc/grafana/grafana-selfsigned.crt
cert_key = /etc/grafana/grafana-selfsigned.key
```

3. **User Management:**
   - Create separate users instead of using admin
   - Implement role-based access control
   - Disable user registration if not needed

System Security

1. **Regular Updates:**

bash

```
# Update system packages
sudo yum update -y  # Amazon Linux
sudo apt update && sudo apt upgrade -y  # Ubuntu

# Update Grafana
sudo yum update grafana  # Amazon Linux
sudo apt update grafana  # Ubuntu
```

2. **Log Monitoring:**

bash

```
# Monitor auth logs
sudo tail -f /var/log/secure  # Amazon Linux
sudo tail -f /var/log/auth.log  # Ubuntu

# Monitor application logs
sudo journalctl -u prometheus -f
sudo journalctl -u grafana-server -f
```

Backup Strategy

1. **Prometheus Data Backup:**

```
bash

# Create backup script
sudo tee /usr/local/bin/prometheus-backup.sh << EOF
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/backup/prometheus"
mkdir -p $BACKUP_DIR

# Stop Prometheus
systemctl stop prometheus

# Create backup
tar -czf $BACKUP_DIR/prometheus_data_$DATE.tar.gz /var/lib/prometheus/

# Start Prometheus
systemctl start prometheus

# Keep only last 7 backups
find $BACKUP_DIR -name "prometheus_data_*.tar.gz" -mtime +7 -delete
EOF

sudo chmod +x /usr/local/bin/prometheus-backup.sh
```

2. **Grafana Dashboard Backup:**

```
bash

# Export all dashboards
curl -H "Authorization: Bearer YOUR_API_KEY" \
  http://localhost:3000/api/search?dashboardIds > dashboards.json
```

3. **Automated Backups:**

```bash
# Add to crontab
sudo crontab -e

# Daily backup at 2 AM
0 2 * * * /usr/local/bin/prometheus-backup.sh
```

Conclusion

You have successfully set up a complete monitoring stack with Prometheus and Grafana on your AWS EC2 Linux instance. This setup provides:

What You've Accomplished:

1. **Prometheus Server:** Collecting and storing time-series metrics
2. **Node Exporter:** Providing detailed system metrics
3. **Grafana:** Beautiful dashboards and visualizations
4. **Custom Dashboards:** Tailored monitoring views
5. **Security Configuration:** Basic security hardening

Key Benefits:

- **Real-time Monitoring:** Monitor your system performance in real-time
- **Historical Data:** Track trends and patterns over time
- **Alerting Capability:** Set up alerts for critical conditions
- **Scalability:** Easy to add more targets and exporters
- **Cost-effective:** Open-source solution with professional features

Next Steps:

1. **Add More Targets:** Monitor additional servers, applications, or services
2. **Set Up Alerting:** Configure alert rules and notification channels
3. **Advanced Dashboards:** Create specialized dashboards for specific use cases
4. **High Availability:** Consider clustering for production environments
5. **Integration:** Connect with other tools like PagerDuty, Slack, or email

Monitoring Best Practices:

- Regular review of dashboards and alerts
- Capacity planning based on historical data
- Documentation of custom metrics and dashboards
- Regular backup of configuration and data
- Stay updated with latest versions and security patches

Support and Resources:

- **Prometheus Documentation:** https://prometheus.io/docs/
- **Grafana Documentation:** https://grafana.com/docs/
- **Community Forums:** Active communities for both tools
- **GitHub Issues:** Report bugs and request features

Your monitoring stack is now ready for production use. Monitor your infrastructure effectively and proactively address issues before they impact your users.

Document Version: 1.0
Last Updated: August 2025
Total Pages: 32

This guide provides a comprehensive foundation for monitoring with Prometheus and Grafana. Customize the configuration based on your specific requirements and environment.